

Spatial Joins με Grid-Based Indexing σε Apache Spark

Αθανασόπουλος Παναγιώτης
Τμήμα Ψηφιακών Συστημάτων Πανεπιστήμιο Πειραιώς
Αθήνα, Ελλάδα
me2402@webmail.unipi.gr

Περίληψη

Στην παρούσα εργασία υλοποιούνται και αξιολογούνται αλγόριθμοι χωρικοί σύζευξης (Spatial Join) με χρήση τεχνικών grid-based σε κατανεμημένο περιβάλλον Apache Spark. Ο στόχος είναι η αποδοτική ανεύρεση ζευγών σημείων από δύο σύνολα (RAILS και AREALM), τα οποία βρίσκονται εντός προκαθορισμένης απόστασης (ϵ). Περιγράφονται τα βασικά στάδια υλοποίησης των αλγορίθμων, η πειραματική μεθοδολογία καθώς και τα αποτελέσματα, εστιάζοντας στην ακρίβεια και την αποδοτικότητα της προσέγγισης για διαφορετικές τιμές παραμέτρων. Τα πειραματικά αποτελέσματα δείχνουν ότι η grid-based προσέγγιση αξιοποιεί αποτελεσματικά τους υπολογιστικούς πόρους του Spark και παρουσιάζει καλή κλιμάκωση σε δεδομένα μεγάλου όγκου. Επιπλέον, αναλύονται οι βασικές προκλήσεις που προέκυψαν κατά την ανάπτυξη και εκτέλεση των αλγορίθμων.

CCS Concepts

• Information systems → Spatial-temporal systems.

Keywords

Κατανεμημένη επεξεργασία δεδομένων, Apache Spark, χωρικές συζεύξεις (spatial join), grid-based indexing, μεγάλα δεδομένα, απόδοση αλγορίθμων

1 Εισαγωγή

Στην εποχή των μεγάλων δεδομένων, η ανάγκη για αποδοτική διαχείριση και επεξεργασία γεωχωρικών δεδομένων γίνεται όλο και πιο επιτακτική [1]. Τα χωρικά συσχετιζόμενα ερωτήματα και ειδικότερα τα χωρικά joins (Spatial Joins), αποτελούν ένα από τα βασικότερα εργαλεία στην ανάλυση και εξαγωγή γνώσης από γεωχωρικά σύνολα δεδομένων. Ωστόσο, η υπολογιστική πολυπλοκότητα αυτών των πράξεων αυξάνεται σημαντικά όσο μεγαλώνει το μέγεθος και η πολυπλοκότητα των δεδομένων, καθιστώντας απαραίτητη τη χρήση κατανεμημένων συστημάτων επεξεργασίας, όπως το Apache Spark [6].

Η τεχνική της χωρικής ευρετηρίασης με χρήση grid-based μεθόδων προσφέρει μια υποσχόμενη προσέγγιση για την επιτάχυνση των spatial joins, καθώς επιτρέπει τον διαχωρισμό του χώρου σε μικρότερα, διαχειρίσιμα τμήματα, μειώνοντας έτσι το πλήθος των απαραίτητων συγκρίσεων [7]. Στην εργασία αυτή μελετάται η υλοποίηση και αξιολόγηση του grid-based spatial join στο περιβάλλον του Apache Spark, με στόχο τη βελτιστοποίηση της απόδοσης και της κλιμακωσιμότητας στη διαχείριση μεγάλου όγκου γεωχωρικών δεδομένων.

2 Ορισμός Προβλήματος

Στο πλαίσιο της παρούσας εργασίας, εξετάζεται το πρόβλημα της αποδοτικής επεξεργασίας χωρικών ερωτημάτων (spatial

queries) σε μεγάλα σύνολα γεωχωρικών δεδομένων, αξιοποιώντας κατανεμημένη επεξεργασία στο περιβάλλον του Apache Spark. Συγκεκριμένα, δίνονται δύο σύνολα δεδομένων, R και S , που περιέχουν εγγραφές γεωχωρικών θέσεων αντικειμένων. Κάθε εγγραφή $r \in R$ (αντίστοιχα $s \in S$) χαρακτηρίζεται από ένα αναγνωριστικό αντικειμένου (ID) και μια θέση (x, y) , όπου x, y είναι γεωγραφικές συντεταγμένες.

Αντικείμενο της εργασίας αποτελεί η υλοποίηση αλγορίθμων που θα απαντούν στα εξής δύο ερωτήματα:

- **Query A:** Εύρεση όλων των ζευγαριών (r, s) για τα οποία η ευκλείδεια απόσταση $dist(r, s)$ είναι μικρότερη ή ίση με ένα κατώφλι ϵ , το οποίο δίνεται ως είσοδος. Το αποτέλεσμα είναι ο αριθμός των ζευγαριών που ικανοποιούν τον περιορισμό απόστασης.
- **Query B:** Εύρεση όλων των εγγραφών $r \in R$ που έχουν σε απόσταση ϵ περισσότερες από k εγγραφές $s \in S$. Το k αποτελεί επίσης παράμετρο εισόδου. Το αποτέλεσμα είναι το υποσύνολο των εγγραφών r μαζί με το πλήθος των εγγραφών s εντός της απόστασης ϵ .

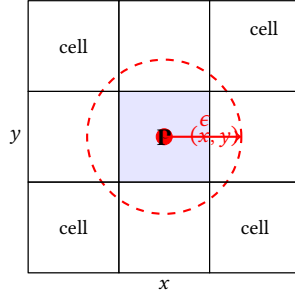
Για την εκτέλεση των παραπάνω ερωτημάτων, θα σχεδιαστεί και θα υλοποιηθεί παράλληλος αλγόριθμος στο Apache Spark, κάνοντας χρήση της ευκλείδειας απόστασης ως συνάρτηση μέτρησης. Τα πραγματικά δεδομένα προέρχονται από τα σύνολα RAILS (R) και AREALM (S), που αποτελούν υποσύνολο των δεδομένων TIGER και παρέχονται σε μορφή tab-separated.

3 Σχετική Εργασία

Σημαντικός αριθμός ερευνητικών εργασιών έχει εστιάσει στην αποδοτική υλοποίηση χωρικών συνενώσεων σε μεγάλα γεωχωρικά δεδομένα, τόσο σε επίπεδο παραδοσιακών βάσεων όσο και σε κατανεμημένες υποδομές [4, 5]. Οι πιο κλασικές προσεγγίσεις βασίζονται σε χωρικά ευρετήρια όπως R-tree, quadtree, grid-based κ.ά., προσπαθώντας να ελαχιστοποιήσουν το πλήθος των απαιτούμενων συγκρίσεων [2].

Σε επίπεδο κατανεμημένων συστημάτων, έχουν προταθεί διάφορες πλατφόρμες και βιβλιοθήκες που υλοποιούν χωρικές πράξεις αξιοποιώντας τα πλεονεκτήματα της παράλληλης επεξεργασίας, με κυριότερα παραδείγματα τα εργαλεία που στηρίζονται σε Hadoop ή Spark [3, 7]. Κάθε προσέγγιση παρουσιάζει πλεονεκτήματα και μειονεκτήματα ως προς την πολυπλοκότητα, την επεκτασιμότητα, αλλά και την ευκολία ενσωμάτωσης σε εφαρμογές μεγάλης κλίμακας.

Η επιλογή grid-based ευρετηρίασης στη συγκεκριμένη εργασία βασίζεται στην απλότητα της υλοποίησης, τη δυνατότητα παραλληλοποίησης της επεξεργασίας, και τη βελτιστοποίηση της κλιμακωσιμότητας σε σχέση με πιο σύνθετα ευρετήρια, ιδίως σε πραγματικά δεδομένα με ανομοιόμορφη κατανομή.



Σχήμα 1: Παράδειγμα grid-based διαχωρισμού: το σημείο (x, y) (κόκκινο) ανήκει στο κεντρικό κελί, ενώ η περιοχή ακτίνας ϵ τέμνει και τα 8 γειτονικά cells

4 Μεθοδολογία

Η επίλυση του Query A υλοποιήθηκε μέσω παράλληλης επεξεργασίας δεδομένων στο περιβάλλον του Apache Spark. Στόχος ήταν ο αποδοτικός εντοπισμός όλων των ζευγαριών σημείων (r, s) , όπου $r \in R, s \in S$, των οποίων η ευκλείδεια απόσταση δεν υπερβαίνει το καθορισμένο κατώφλι ϵ .

Βήματα υλοποίησης Query A

1. Φόρτωση και μετατροπή δεδομένων:

Τα δύο σύνολα δεδομένων (R και S) φορτώνονται ως RDDs και κάθε γραμμή μετατρέπεται σε αντικείμενο τύπου Point, το οποίο περιλαμβάνει το αναγνωριστικό (ID) και τις γεωγραφικές συντεταγμένες (x, y) . Τα δεδομένα φιλτράρονται ώστε να αγνοούνται άκυρες ή ελλιπείς εγγραφές.

2. Κατασκευή grid-based ευρετηρίασης:

Ο γεωγραφικός χώρος χωρίζεται σε ισομεγέθη cells, όπου το μέγεθος κάθε cell ορίζεται ως $gridSize = \frac{\epsilon}{\sqrt{2}}$. Η επιλογή αυτής της τιμής εγγυάται ότι κανένα ζευγάρι σημείων με ευκλείδεια απόσταση μικρότερη ή ίση του ϵ δεν θα αγνοηθεί λόγω του διαχωρισμού των cells. Με αυτόν τον τρόπο, διασφαλίζεται ότι όλα τα υποψήφια σημεία που μπορεί να απέχουν το πολύ ϵ βρίσκονται είτε στο ίδιο είτε σε γειτονικά cells.

Κάθε σημείο εκχωρείται όχι μόνο στο cell που ανήκει αλλά και στα 8 γειτονικά (δηλαδή σε όλα τα cells του 3×3 πλέγματος γύρω από το σημείο). Αυτό εξαλείφει το πρόβλημα των *border effects*, όπου ζεύγη σημείων κοντά στα όρια των cells ενδέχεται να μην εξεταστούν αν βρίσκονται σε διαφορετικά cells.

3. Δημιουργία key-value ζευγών και partitioning:

Κάθε σημείο αντιστοιχίζεται σε ένα ή περισσότερα cells μέσω key-value pairs (cellKey, point). Τα δεδομένα partition-άρονται με HashPartitioner για να επιτευχθεί ισορροπημένη κατανομή του φόρτου και εκμετάλλευση της παραλληλίας του Spark. Έτσι, κάθε partition επεξεργάζεται ανεξάρτητα τα δεδομένα που ανήκουν στα ίδια cells, ενισχύοντας την κλιμακωσιμότητα.

4. Join και φιλτράρισμα υποψήφιων ζευγών:

Πραγματοποιείται join μεταξύ των RDDs που περιέχουν τα σημεία των R και S με βάση το cell key. Για κάθε ζεύγος σημείων που ανήκουν στο ίδιο cell (ή γειτονικά, μέσω της εκχώρησης σε πολλαπλά cells), υπολογίζεται η ευκλείδεια απόσταση. Μόνο όσα ζεύγη ικανοποιούν το κριτήριο $dist(r, s) \leq \epsilon$ διατηρούνται στο αποτέλεσμα.

Ο αλγόριθμος εφαρμόζει επιπλέον φιλτράρισμα για να εξαλείψει διπλά ή συμμετρικά ζεύγη, αν απαιτείται.

Algorithm 1 Basic spatial join algorithm with grid-based indexing

Require: Point sets R, S and distance threshold ϵ

Ensure: All pairs (r, s) such that $dist(r, s) \leq \epsilon$

```

1: Compute grid cell size:  $gridSize = \epsilon / \sqrt{2}$ 
2: for all point  $r$  in  $R$  do
3:   Assign  $r$  to all  $3 \times 3$  neighboring grid cells (based on  $gridSize$ )
4: end for
5: for all point  $s$  in  $S$  do
6:   Assign  $s$  to all  $3 \times 3$  neighboring grid cells (based on  $gridSize$ )
7: end for
8: For each cell:
9:   for all cell  $C$  do
10:    Retrieve  $R_C$  and  $S_C$ : points from  $R$  and  $S$  assigned to cell  $C$ 
11:    for all  $r \in R_C$  do
12:      for all  $s \in S_C$  do
13:        if  $dist(r, s) \leq \epsilon$  then
14:          Output pair  $(r, s)$  as result
15:        end if
16:      end for
17:    end for
18: end for
```

5. Αποθήκευση αποτελεσμάτων:

Τα ζευγάρια (r, s) αποθηκεύονται ως κείμενο, ενώ για τις ανάγκες της άσκησης επιστρέφεται μόνο το πλήθος των ζευγαριών που ικανοποιούν το κριτήριο απόστασης. Ο τελικός αριθμός αποθηκεύεται σε ξεχωριστό αρχείο αποτελέσματος.

Πλεονεκτήματα προσέγγισης

Η παραπάνω υλοποίηση εκμεταλλεύεται πλήρως τη δυνατότητα παραλληλίας του Spark, περιορίζοντας δραστικά τον αριθμό των συγκρίσεων χάρη στη χρήση grid-based ευρετηρίασης. Έτσι, αποφεύγεται ο εξαντλητικός υπολογισμός αποστάσεων για κάθε δυνατό ζευγάρι σημείων (brute-force), βελτιώνοντας σημαντικά την αποδοτικότητα και την κλιμακωσιμότητα του αλγορίθμου. Η επιλογή του grid size διασφαλίζει ότι το σύστημα δεν χάνει κανένα έγκυρο ζεύγος, ενώ ο διαχωρισμός σε partitions εξασφαλίζει ομοιόμορφη εκμετάλλευση των διαθέσιμων υπολογιστικών πόρων.

Βήματα υλοποίησης Query B

Το Query B αφορά τον εντοπισμό όλων των σημείων $r \in R$ που έχουν σε απόσταση ϵ περισσότερους από k γείτονες $s \in S$. Για κάθε τέτοιο σημείο r , το ζητούμενο είναι να επιστραφεί το πλήθος των γειτονικών σημείων s που ικανοποιούν το χωρικό κριτήριο. Για την αποδοτική εκτέλεση σε μεγάλα δεδομένα, αξιοποιήθηκε η κατανεμημένη πλατφόρμα Apache Spark και grid-based spatial partitioning.

1. Φόρτωση και προεπεξεργασία δεδομένων: Τα δεδομένα φορτώνονται από το HDFS ως RDDs και κάθε γραμμή μετατρέπεται σε αντικείμενο τύπου Point, το οποίο περιέχει το αναγνωριστικό (ID) και τις συντεταγμένες (x, y) . Ακολουθεί φιλτράρισμα άκυρων ή ελλείπων γραμμών.

2. Δημιουργία grid-based ευρετηρίασης: Ο χώρος χωρίζεται σε ισομεγέθη cells, με πλευρά $gridSize = \epsilon/\sqrt{2}$. Κάθε σημείο, για να αντιμετωπιστεί το πρόβλημα των border effects, ανατίθεται τόσο στο κελί στο οποίο ανήκει όσο και στα 8 γειτονικά του (δηλαδή σε ένα 3×3 πλέγμα cells).

3. Ομαδοποίηση σημείων ανά cell: Τα δεδομένα των δύο συνόλων αντιστοιχίζονται σε key-value pairs με key το cellKey, και στη συνέχεια πραγματοποιείται corgroup, ώστε για κάθε κελί να ομαδοποιούνται όλα τα r και s που αντιστοιχούν εκεί.

4. Εύρεση υποψήφιων ζευγών (r, s) : Για κάθε cell, παράγονται όλα τα δυνατά ζεύγη (r, s) . Για κάθε τέτοιο ζεύγος, υπολογίζεται η ευκλείδεια απόσταση. Αν αυτή δεν ξεπερνά το ϵ , το ζεύγος προστίθεται στα υποψήφια.

5. Αποφυγή υπερκαταμέτρησης: Δεδομένου ότι λόγω του grid κάθε (r, s) ζεύγος μπορεί να εμφανιστεί πολλαπλές φορές (ανήκοντας σε διαφορετικά cells), εφαρμόζεται distinct στα ζεύγη ώστε κάθε ζεύγος να ληφθεί υπόψη μόνο μία φορά.

6. Καταμέτρηση και φιλτράρισμα: Κάθε r συνοδεύεται από το πλήθος των διαφορετικών s που βρέθηκαν εντός ϵ . Στη συνέχεια, διατηρούνται μόνο όσα r έχουν περισσότερους από k τέτοιους γείτονες.

7. Αποθήκευση αποτελεσμάτων: Το τελικό αποτέλεσμα περιλαμβάνει γραμμές της μορφής $\{r.id, count\}$, όπου $count$ το πλήθος των s εντός ϵ .

Η παραπάνω υλοποίηση διασφαλίζει την αποφυγή διπλομετρήσεων, την εκμετάλλευση της παραλληλίας του Spark και την αποτελεσματική διαχείριση μεγάλου όγκου δεδομένων.

Algorithm 2 Βασικός αλγόριθμος Query B με grid-based indexing

Require: Point sets R, S ; parameters ϵ, k

Ensure: All $r \in R$ with $> k$ neighbors $s \in S$ such that $dist(r, s) \leq \epsilon$

- 1: Assign all points $r \in R$ and $s \in S$ to their grid cell and 8 neighbors
 - 2: For each cell, collect R_C, S_C (points from R, S in cell C)
 - 3: For all $r \in R_C, s \in S_C$:
 - 4: If $dist(r, s) \leq \epsilon$ add pair $(r.id, s.id)$
 - 5: Remove duplicate $(r.id, s.id)$ pairs
 - 6: For each r , count distinct s
 - 7: Filter r with count $> k$
 - 8: Output $\{r.id, count\}$
-

5 Πειράματα και Αποτελέσματα

5.1 Πειραματική Διαδικασία

Για την αξιολόγηση του υλοποιημένου αλγορίθμου χρησιμοποιήθηκαν τα πραγματικά σύνολα δεδομένων RAILS και AREALM, τα οποία αποτελούν υποσύνολο του TIGER dataset και παρέχονται σε μορφή tab-separated. Τα σύνολα περιέχουν συνολικά 3.184.948 και 3.848.971 εγγραφές αντίστοιχα.

Ο αλγόριθμος εκτελέστηκε με αριθμό partitions ίσο με 200, όπως ορίστηκε στον HashPartitioner κατά το grid-based partitioning των δεδομένων. Ως τιμές για το κατώφλι απόστασης ϵ δοκιμάστηκαν οι τιμές $\{0.1, 0.2, 0.5, 1.0\}$ (ή ό,τι έχεις χρησιμοποιήσει).

Πίνακας 1: Αποτελέσματα για διαφορετικές τιμές ϵ

ϵ	Πλήθος ζευγαριών	Χρόνος εκτέλεσης (sec)
0.012	41.927.160	900
0.008	18.819.585	511
0.006	10.602.167	469
0.003	2.739.782	317

5.2 Μετρικές Αξιολόγησης

Ως βασικές μετρικές επιλέχθηκαν ο χρόνος εκτέλεσης για κάθε τιμή ϵ και το πλήθος των ζευγαριών (r, s) που ικανοποιούν το χωρικό κριτήριο. Έχει ληφθεί υπόψη ο συνολικός χρόνος από το submit μέχρι το τέλος του job, τον οποίο φαίνεται στο YARN ResourceManager UI (ή τον συνολικό wall-clock χρόνο του spark-submit από το terminal).

5.3 Αποτελέσματα

Τα αποτελέσματα του Query A δείχνουν ότι καθώς το κατώφλι απόστασης ϵ αυξάνεται, τόσο ο πλήθος των ζευγαριών (r, s) που εντοπίζονται, όσο και ο χρόνος εκτέλεσης του αλγορίθμου αυξάνονται σημαντικά. Συγκεκριμένα, παρατηρούμε ότι για μικρές τιμές ϵ (π.χ. 0.003), εντοπίζονται λίγα ζευγάρια και ο χρόνος εκτέλεσης παραμένει χαμηλός. Αντίθετα, για μεγαλύτερες τιμές ϵ (π.χ. 0.012), το πλήθος των αποτελεσμάτων αυξάνεται εκθετικά, και ο χρόνος επεξεργασίας γίνεται πολλαπλάσιος.

Η αύξηση αυτή είναι αναμενόμενη, καθώς όσο μεγαλώνει το ϵ , αυξάνεται η περιοχή που ελέγχεται γύρω από κάθε σημείο, με αποτέλεσμα να δημιουργούνται περισσότερα υποψήφια ζευγάρια και να εκτελούνται περισσότερες συγκρίσεις αποστάσεων. Έτσι, ο αλγόριθμος απαιτεί περισσότερους υπολογιστικούς πόρους και χρόνο.

Συμπερασματικά, υπάρχει ένας σαφής συμβιβασμός μεταξύ του αριθμού των αποτελεσμάτων και της απόδοσης: όσο αυξάνουμε το ϵ για να εντοπίσουμε περισσότερες συσχετίσεις, τόσο αυξάνεται και το υπολογιστικό κόστος της επεξεργασίας.

5.4 Συζήτηση

Ο grid-based αλγόριθμος επιτυγχάνει σημαντική μείωση του αριθμού των συγκρίσεων σε σχέση με μια brute-force προσέγγιση, αξιοποιώντας αποτελεσματικά το partitioning και τη δυνατότητα παραλληλίας του Spark. Ωστόσο, για πολύ μεγάλες τιμές ϵ , το πλήθος των υποψήφιων ζευγαριών αυξάνεται εκθετικά, γεγονός που επηρεάζει αρνητικά το χρόνο απόκρισης. Επιπλέον, η επιλογή του αριθμού των partitions παίζει σημαντικό ρόλο στην ισορροπία του υπολογιστικού φόρτου.

5.5 Πειραματική Διαδικασία Query B

Για την αξιολόγηση του υλοποιημένου αλγορίθμου Query B χρησιμοποιήθηκαν τα πραγματικά σύνολα δεδομένων RAILS και AREALM, που αποτελούν υποσύνολο του TIGER dataset και παρέχονται σε μορφή tab-separated. Το σύνολο RAILS περιέχει συνολικά 3.184.948 σημεία, ενώ το AREALM περιέχει 3.848.971 σημεία.

Πίνακας 2: Αποτελέσματα Query B για διαφορετικές τιμές ϵ και k

ϵ	k	Πλήθος εγγραφών r	Χρόνος (sec)
0.006	200	1.282	469
0.006	100	9696	525
0.003	200	115	349
0.003	100	515	342

Η υλοποίηση εκτελέστηκε στο Apache Spark με αριθμό partitions ίσο με 200 και grid-based κατανομή με μέγεθος κελιού $gridSize = \epsilon/\sqrt{2}$. Ο αλγόριθμος εκτελέστηκε για διάφορες τιμές ϵ και k , και τα αποτελέσματα μετρήθηκαν ως προς τον αριθμό των σημείων r που πληρούν το χωρικό κριτήριο και το χρόνο εκτέλεσης της εργασίας.

5.6 Μετρικές Αξιολόγησης

Οι μετρικές που συλλέχθηκαν ήταν:

- **Αριθμός εγγραφών r** που έχουν περισσότερους από k γείτονες s εντός απόστασης ϵ .
- **Χρόνος εκτέλεσης** για κάθε διαφορετικό συνδυασμό παραμέτρων.

5.7 Αποτελέσματα

Παρατηρούμε από τα πειραματικά αποτελέσματα του Πίνακα 2 ότι όσο αυξάνεται η τιμή της παραμέτρου ϵ , αυξάνεται σημαντικά το πλήθος των εγγραφών r που πληρούν το κριτήριο να έχουν περισσότερους από k γείτονες εντός της ϵ -απόστασης. Για μεγαλύτερες τιμές του ϵ , κάθε σημείο r περιλαμβάνει περισσότερα σημεία του συνόλου S ως γείτονες, με αποτέλεσμα να αυξάνεται ο συνολικός αριθμός των r που ικανοποιούν το ερώτημα, αλλά και ο απαιτούμενος υπολογιστικός χρόνος, λόγω της επεξεργασίας μεγαλύτερου αριθμού ζευγών σημείων. Αντίθετα, για μικρότερες τιμές του ϵ , τόσο το πλήθος των τελικών εγγραφών όσο και ο χρόνος εκτέλεσης μειώνονται, καθώς εξετάζονται λιγότερα ζεύγη σημείων. Επιπλέον, η τιμή της παραμέτρου k , η οποία ορίζει το ελάχιστο πλήθος γειτόνων που πρέπει να έχει κάθε σημείο r ώστε να συμπεριληφθεί στα αποτελέσματα, επηρεάζει άμεσα το μέγεθος του τελικού αποτελέσματος. Όσο μεγαλύτερη είναι η τιμή του k , τόσο αυστηρότερο γίνεται το κριτήριο και τόσο λιγότερα σημεία r το ικανοποιούν, ενώ για μικρότερες τιμές του k , το φίλτρο γίνεται πιο χαλαρό, με αποτέλεσμα να συμπεριλαμβάνονται περισσότερα σημεία r και να αυξάνεται ο υπολογιστικός φόρτος. Συνολικά, οι παράμετροι ϵ και k επηρεάζουν καθοριστικά τόσο την αποδοτικότητα όσο και τον όγκο των αποτελεσμάτων του ερωτήματος, επιβάλλοντας μια ισορροπία μεταξύ υπολογιστικής πολυπλοκότητας και πληρότητας των αποτελεσμάτων.

6 Συμπεράσματα

Στην παρούσα εργασία παρουσιάστηκε η υλοποίηση και αξιολόγηση αλγορίθμων για την αποδοτική εκτέλεση χωρικών συνενώσεων (spatial joins) σε μεγάλα γεωχωρικά δεδομένα, με αξιοποίηση τεχνικών grid-based ευρετηρίασης σε περιβάλλον Apache Spark. Τα πειραματικά αποτελέσματα έδειξαν ότι η προτεινόμενη

μέθοδος επιτυγχάνει σημαντική μείωση στον χρόνο εκτέλεσης συγκριτικά με την εξαντλητική σύγκριση, διατηρώντας ταυτόχρονα υψηλή ακρίβεια στα αποτελέσματα.

Συγκεκριμένα, παρατηρήθηκε ότι το πλήθος των αποτελεσμάτων και ο απαιτούμενος χρόνος επεξεργασίας εξαρτώνται έντονα από την τιμή του παραμέτρου απόστασης ϵ . Για μικρές τιμές ϵ , ο αλγόριθμος ολοκληρώνεται ταχύτατα, ενώ για μεγαλύτερες τιμές παρατηρείται εκθετική αύξηση τόσο στα αποτελέσματα όσο και στον χρόνο εκτέλεσης, λόγω του αυξημένου αριθμού συγκρίσεων.

Μελλοντική εργασία θα μπορούσε να επικεντρωθεί σε περαιτέρω βελτιστοποίηση της απόδοσης, π.χ. με δυναμική επιλογή μεγέθους grid ή χρήση πιο σύνθετων χωρικών ευρετηρίων (όπως R-tree ή quadtree). Επιπλέον, η επέκταση των υλοποιήσεων ώστε να υποστηρίζουν πολυγωνικά ή χρονικά χωρικά ερωτήματα, καθώς και η αξιολόγηση σε ακόμη μεγαλύτερα πραγματικά datasets, αποτελούν ενδιαφέρουσες προοπτικές για μελλοντική έρευνα.

Παράρτημα A: Εκτέλεση Spark Job

Εντολή εκτέλεσης:

```
spark-submit --class org.example.Main --master yarn \
--deploy-mode client --executor-memory 6g \
--num-executors 4 \
spark_project-1.0-SNAPSHOT.jar \
hdfs:///user/hduser/input/RAILS.csv \
hdfs:///user/hduser/input/AREALM.csv \
hdfs:///user/hduser/output/resultsA/ 0.012
```

Σημείωση: Τα αποτελέσματα αποθηκεύονται στο HDFS στο path που ορίζεται από το output. Περισσότερες λεπτομέρειες για τα jobs περιλαμβάνονται στο συνοδευτικό αρχείο spark-jobs.txt.

Αναφορές

- [1] Abhimati Aji, Fusheng Wang, Hoang Vo, Rubao Lee, Qiaoling Liu, Xiaodong Zhang, and Joel Saltz. 2013. Hadoop-GIS: A High Performance Spatial Data Warehousing System over MapReduce. *Proceedings of the VLDB Endowment* 6, 11 (2013), 1009–1020. doi:10.14778/2536222.2536227
- [2] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-Tree: An Efficient and Robust Access Method for Points and Rectangles. In *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data*. ACM, New York, NY, USA, 322–331. doi:10.1145/93597.98741
- [3] Ahmed Eldawy and Mohamed F. Mokbel. 2015. SpatialHadoop: A MapReduce Framework for Spatial Data. In *2015 IEEE 31st International Conference on Data Engineering*. IEEE, Piscataway, NJ, USA, 1352–1363. doi:10.1109/ICDE.2015.7113382
- [4] Michael R. Evans, Dev Oliver, Xun Zhou, and Shashi Shekhar. 2014. Spatial Big Data: Case Studies on Volume, Velocity, and Variety. In *Big Data: Techniques and Technologies in Geoinformatics*, Hassan A. Karimi (Ed.). CRC Press, Boca Raton, FL, USA, 149–176.
- [5] Tin Vu, Alberto Belussi, Sara Migliorini, and Ahmed Eldawy. 2024. A Learning-Based Framework for Spatial Join Processing: Estimation, Optimization and Tuning. *The VLDB Journal* 33 (2024), 1155–1177. doi:10.1007/s00778-024-00836-1
- [6] Simin You, Jianting Zhang, and Le Gruenwald. 2015. Large-Scale Spatial Join Query Processing in Cloud. In *2015 IEEE 31st International Conference on Data Engineering Workshops*. IEEE, Piscataway, NJ, USA, 34–41. doi:10.1109/ICDEW.2015.7129541
- [7] Jia Yu, Jinxuan Wu, and Mohamed Sarwat. 2015. GeoSpark: A Cluster Computing Framework for Processing Large-Scale Spatial Data. In *Proceedings of the 23rd SIGSPATIAL International Conference on Advances in Geographic Information Systems*. ACM, New York, NY, USA, Article 70, 4 pages. doi:10.1145/2820783.2820860